Human Verification: `Containers.DContFunctor`

Covariant Functors of Directed Containers

Machine-generated from `lean/DContFunctor.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Containers.DContFunctor` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean. The purpose is to let a human verifier confirm that the formalisation captures the intended mathematics.

Contents

1 Overview **1**

2 The `DContFunctor` structure **2**
2.1 Structure: `DContFunctor` 2

3 Identity and composition **3**
3.1 Definition: `DContFunctor.id` 3
3.2 Lemma: `DContFunctor.onPos_transport` 3
3.3 Lemma: `DContFunctor.transport_trans` 3
3.4 Definition: `DContFunctor.comp` 4

4 Category laws for `DContFunctor` **4**
4.1 Theorems: identity and associativity 4

5 Translation conventions **5**

1 Overview

This file defines **covariant functors of directed containers** — the morphism notion needed for Construction *con:sourcing* in the supply-chains paper. Contrast with `DContMorphism` (in `Containers.DContCat`), whose position map is *contravariant* and corresponds to a *cofunctor* of the induced categories.

A `DContFunctor C D` is a *functor* $\mathcal{C}(C) \to \mathcal{C}(D)$:

- the shape map $f : S_C \to S_D$ acts on objects *covariantly*;
- the position map $f_s^\flat : P_C(s) \to P_D(f(s))$ acts on morphisms *covariantly* (positions go *forward*);
- F0 says the codomain of $f_s^\flat(p)$ is $f(s \downarrow p)$;
- F1 says roots (identities) are preserved;

- F2 says shifts (compositions) are preserved, with a transport along F0.

The file also establishes that `DContFunctor` forms a category (identity, composition, and the three category laws, all by `rfl`).

2 The `DContFunctor` structure

2.1 Structure: `DContFunctor`

Definition 2.1 (Covariant functor of directed containers)**.** A **covariant directed-container functor** $F : C \to D$ consists of:

- a *covariant* shape map $f : S_C \to S_D$;
- a *covariant* position map $f_s^\flat : P_C(s) \to P_D(f(s))$;

satisfying:

- **F0:** $f(C{\downarrow_s}p) = D{\downarrow_{f(s)}} f_s^\flat(p)$ (codomain preserved);
- **F1:** $f_s^\flat(o_C(s)) = o_D(f(s))$ (roots preserved);
- **F2:** $f_s^\flat(p \oplus_C q) = f_s^\flat(p) \oplus_D (\mathrm{F0} \triangleright f^\flat_{C\downarrow_s p}(q))$ (shifts preserved, with transport along F0).

Lean.

```
structure DContFunctor (C D : DirectedContainer) where
  onShapes : C.Shape → D.Shape
  onPos : (s : C.Shape) → C.Pos s → D.Pos (onShapes s)
  f0 : ∀ (s : C.Shape) (p : C.Pos s),
    onShapes (C.sub s p) = D.sub (onShapes s) (onPos s p)
  f1 : ∀ s : C.Shape, onPos s (C.root s) = D.root (onShapes s)
  f2 : ∀ (s : C.Shape) (p : C.Pos s) (q : C.Pos (C.sub s p)),
    onPos s (C.shift s p q)
      = D.shift (onShapes s) (onPos s p)
          (f0 s p ▸ onPos (C.sub s p) q)
```

Haskell.

```
data DContFunctor c d = DContFunctor
  { onShapes :: Shape c → Shape d          -- covariant
  , onPos    :: Shape c → Pos c → Pos d -- covariant (positions go
    forward)
  -- F0: onShapes (sub_C s p) = sub_D (onShapes s) (onPos s p)
  -- F1: onPos s (root_C s) = root_D (onShapes s)
  -- F2: onPos s (shift_C s p q)
  --       = shift_D (onShapes s) (onPos s p)
  --           (onPos (sub_C s p) q)
  --   (transport along F0 is suppressed in Haskell)
  }
```

Fidelity note. The key difference from `DContMorphism`: `onPos` maps $P_C(s)$ to $P_D(f(s))$ (forward), whereas in a `DContMorphism`, `onPos` maps $P_D(f(s))$ to $P_C(s)$ (backward). In F2, the inner position $f^\flat_{C\downarrow s}(q)$ lives over $f(C{\downarrow_s}p)$, but D.shift requires it over $D{\downarrow_{f(s)}} f_s^\flat(p)$; the transport F0$\triangleright$ bridges these two equal shapes. This transport is invisible in Haskell.

3 Identity and composition

3.1 Definition: `DContFunctor.id`

Definition 3.1 (Identity functor)**.** The identity covariant functor on C: both maps are id. All three laws hold by rfl (the F2 transport is along rfl and vanishes definitionally).

Lean.
```
def DContFunctor.id (C : DirectedContainer) : DContFunctor C C where
  onShapes s := s
  onPos _ p := p
  f0 _ _ := rfl
  f1 _ := rfl
  f2 _ _ _ := rfl
```

Haskell.
```
idDCF :: DContFunctor c c
idDCF = DContFunctor id (\_ p → p)
```

3.2 Lemma: `DContFunctor.onPos_transport`

Proposition 3.2 (Transport commutes with the position map)**.** Let $F : C \to D$ and let $e : a = b$ be a shape equality in C. Then

$$F.\mathrm{onPos}_b(e \triangleright p) \;=\; \mathrm{congrArg}(F.\mathrm{onShapes}, e) \triangleright F.\mathrm{onPos}_a(p).$$

That is, pushing a position forward after transporting along e equals transporting the image along the image equality.

Lean.
```
theorem onPos_transport {C D : DirectedContainer} (phi : DContFunctor
    C D)
    {a b : C.Shape} (e : a = b) (p : C.Pos a) :
    phi.onPos b (e [> p) = congrArg phi.onShapes e [> phi.onPos a p
```

Haskell.
```
-- Trivial in Haskell: transport is identity.
-- onPos b p = onPos a p  (since a = b implies same type)
```

3.3 Lemma: `DContFunctor.transport_trans`

Proposition 3.3 (Two transports equal one)**.** For any type family $P : \alpha \to \mathbf{Type}$, equalities $h_1 : x = y$ and $h_2 : y = z$, and element $p : P(x)$:

$$h_2 \triangleright (h_1 \triangleright p) \;=\; (h_1 \cdot h_2) \triangleright p.$$

Lean.
```
theorem transport_trans {alpha : Type} {P : alpha → Type} {x y z :
    alpha}
    (h1 : x = y) (h2 : y = z) (p : P x) :
    h2 [> h1 [> p = (h1.trans h2) [> p
```

Haskell.
```
-- Trivial in Haskell (transport is identity).
```

3.4 Definition: `DContFunctor.comp`

Definition 3.4 (Composition of covariant functors)**.** Given $F : C \to D$ and $G : D \to E$, their composite $G \circ F : C \to E$ has:

$$\begin{aligned} \text{onShapes}(s) &= G.\text{onShapes}(F.\text{onShapes}(s)), \\ \text{onPos}_s(p) &= G.\text{onPos}_{F.\text{onShapes}(s)}(F.\text{onPos}_s(p)). \end{aligned}$$

Both maps compose forwards. F0 is the composite of the factors' F0s; F1 uses two rewrites; F2 uses both factors' F2s followed by the two transport lemmas.

Lean.

```
def DContFunctor.comp {C D E : DirectedContainer}
    (phi : DContFunctor C D) (psi : DContFunctor D E) : DContFunctor C
        E where
  onShapes s := psi.onShapes (phi.onShapes s)
  onPos s p := psi.onPos (phi.onShapes s) (phi.onPos s p)
  f0 s p := (congrArg psi.onShapes (phi.f0 s p)).trans
    (psi.f0 (phi.onShapes s) (phi.onPos s p))
  f1 s := by rw [phi.f1, psi.f1]
  f2 s p q := by
    rw [phi.f2, psi.f2, psi.onPos_transport (phi.f0 s p),
       transport_trans]
```

Haskell.

```
compDCF :: DContFunctor c d → DContFunctor d e → DContFunctor c e
compDCF phi psi = DContFunctor
  { onShapes = onShapes psi . onShapes phi
  , onPos    = \s p → onPos psi (onShapes phi s) (onPos phi s p)
  }
```

4 Category laws for `DContFunctor`

4.1 Theorems: identity and associativity

Theorem 4.1 (Left unit)**.** For any $F : C \to D$, $\text{comp}(\text{id}_C, F) = F$ (by `rfl`).

Theorem 4.2 (Right unit)**.** For any $F : C \to D$, $\text{comp}(F, \text{id}_D) = F$ (by `rfl`).

Theorem 4.3 (Associativity)**.** For $F : B \to C$, $G : C \to D$, $H : D \to E$:

$$(H \circ G) \circ F \;=\; H \circ (G \circ F) \quad \text{(by } \texttt{rfl}\text{)}.$$

Lean.

```
theorem DContFunctor.id_comp {C D : DirectedContainer}
    (phi : DContFunctor C D) : comp (id C) phi = phi

theorem DContFunctor.comp_id {C D : DirectedContainer}
    (phi : DContFunctor C D) : comp phi (id D) = phi

theorem DContFunctor.comp_assoc {B C D E : DirectedContainer}
    (phi : DContFunctor B C) (psi : DContFunctor C D) (chi :
       DContFunctor D E) :
    comp (comp phi psi) chi = comp phi (comp psi chi)
```

Haskell.

```
-- prop_id_comp phi = compDCF idDCF phi == phi
-- prop_comp_id phi = compDCF phi idDCF == phi
-- prop_assoc phi psi chi =
--   compDCF (compDCF phi psi) chi == compDCF phi (compDCF psi chi)
```

5 Translation conventions

1. **Covariant vs. contravariant.** `DContFunctor.onPos` maps $P_C(s) \to P_D(f(s))$ (forward); `DContMorphism.onPos` maps $P_D(f(s)) \to P_C(s)$ (backward). This is the defining distinction between functors and cofunctors of directed containers.
2. **F2 transport.** The inner position in F2 must be transported along F0 before `D.shift` can consume it. All three category laws (`rfl`) confirm that these transports compose cleanly.
3. **Design choice: direct definition.** `DContFunctor` is defined directly on containers (not via `toSmallCat`), avoiding the round-trip gap and keeping transports rfl-cheap for containers like `network G` whose `sub` is a projection.
4. **Proof bodies omitted.** Only statements are translated.